



LAPORAN PRAKTIKUM

TUGAS 1 : RESTFUL API Murni dengan Express.js

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : PUTRI REDA MAULIDIA

NIM : 2428240086

Kelas : SI5B

Nomor Topik : 21

Nama Topik : Kebencanaan (Laporan Bencana)

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	PUTRI REDA MAULIDIA
NPM	2428240086
Kelas	SI5B
Pertemuan Ke-	6
Topik / Judul Praktikum	Topik 21 : Kebencanaan (Laporan Bencana) / Tugas 1: RESTful API Murni dengan Express.js
Tanggal Pelaksanaan	6 Oktober 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membuat sebuah RESTful API menggunakan Node.js dan Express.js.
2. Menerapkan routing berdasarkan konsep REST menggunakan method GET, POST, PUT, dan DELETE.
3. Mengolah request dari client serta memberikan response dalam format JSON.
4. Menerapkan validasi data dan penggunaan HTTP status code seperti 200, 201, 400, dan 404.
5. Membuat fitur pencarian data menggunakan query parameter.
6. Melakukan pengujian API secara lokal menggunakan Thunder Client serta menyiapkan API untuk deployment menggunakan Vercel.

II. Dasar Teori

1. REST dan Resource

REST atau Representational State Transfer merupakan pendekatan arsitektur yang digunakan dalam komunikasi antara client dan server melalui HTTP. Dalam REST API, data yang dikelola server direpresentasikan sebagai sebuah resource.

Pada praktikum ini, resource yang digunakan adalah **disaster-reports**. Resource tersebut digunakan untuk menyimpan informasi mengenai laporan bencana. Endpoint utama yang digunakan adalah:

`/disaster-reports`

Sedangkan untuk mengambil data tertentu digunakan parameter ID, misalnya:

`/disaster-reports/1`

Penggunaan resource membuat struktur API lebih teratur karena URL menunjukkan objek yang sedang dikelola.

2. HTTP Method

REST API memanfaatkan beberapa HTTP method untuk menentukan tindakan yang dilakukan terhadap resource.

1. **GET** digunakan untuk mengambil data. Pada project ini, GET dapat digunakan untuk mengambil seluruh laporan bencana, satu laporan berdasarkan ID, maupun data yang telah difilter.
2. **POST** digunakan untuk menambahkan laporan bencana baru ke dalam resource.

3. **PUT** digunakan untuk memperbarui data laporan berdasarkan ID tertentu.

4. **DELETE** digunakan untuk menghapus laporan berdasarkan ID.

Pembagian fungsi tersebut membuat komunikasi antara client dan server lebih mudah dipahami.

3. HTTP Status Code

HTTP status code digunakan untuk menunjukkan hasil dari request yang dikirimkan client. Beberapa status code yang diterapkan pada project ini yaitu:

- **200 OK** menunjukkan request berhasil.
 - **201 Created** menunjukkan data baru berhasil dibuat.
 - **400 Bad Request** menunjukkan data yang dikirim client tidak memenuhi validasi.
 - **404 Not Found** menunjukkan data atau endpoint yang diminta tidak ditemukan.
-

Penggunaan status code membantu client mengetahui kondisi request tanpa harus melihat isi response secara keseluruhan.

4. JSON

JSON atau JavaScript Object Notation merupakan format pertukaran data yang banyak digunakan dalam REST API. Format ini menggunakan struktur pasangan key dan value sehingga mudah dibaca dan diproses oleh aplikasi.

Contoh data laporan bencana:

```
{
```

```
"id": 1,  
  
"jenisBencana": "banjir",  
  
"lokasi": "Kec. Seberang Ulu I",  
  
"tanggal": "2026-09-20",  
  
"tingkatKeparahan": "sedang",  
  
"jumlahPengungsi": 120  
  
}
```

Pada project ini seluruh response API dikirimkan menggunakan format JSON.

5. Express.js

Express.js merupakan framework berbasis Node.js yang digunakan untuk membangun aplikasi server dan REST API. Express menyediakan fasilitas untuk membuat route, membaca request, mengatur response, dan menggunakan middleware.

Salah satu middleware yang digunakan pada project ini adalah:

```
app.use(express.json());
```

Middleware tersebut memungkinkan server membaca data JSON yang dikirim melalui request body.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime untuk menjalankan JavaScript pada sisi server
2	Visual Studio Code	Aplikasi editor yang digunakan untuk menulis, mengedit, serta mengelola berbagai file dalam project pemrograman.
3	Express.js	Framework yang digunakan untuk membangun RESTful API
4	Thunder Client	Digunakan untuk menguji endpoint REST API
5	GitHub Desktop	Digunakan untuk mengelola repository dan commit
6	GitHub	Tempat penyimpanan source code
7	Vercel	Platform untuk deployment API

IV. Langkah Kerja

1. Project dibuat menggunakan Node.js dan dikelola melalui GitHub Desktop serta Visual Studio Code. Setelah project dibuka di Visual Studio Code, terminal digunakan untuk menjalankan perintah: **npm init -y**. Perintah tersebut digunakan untuk membuat konfigurasi awal Node.js melalui file package.json.

2. Selanjutnya memasang Express.js. Express.js dipasang menggunakan: **npm install express**. Selain itu, nodemon dipasang sebagai dependency pengembangan: **npm install --save-dev nodemon**

3. Bagian script pada package.json diatur agar project dapat dijalankan menggunakan:

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js"  
}
```

File utama aplikasi ditentukan sebagai:

```
"main": "app.js"
```

4. Membuat Struktur Project dengan File yang digunakan dalam project antara lain:

.gitignore

README.md

app.js

package.json

package-lock.json

vercel.json

Data laporan bencana disimpan sementara dalam array di dalam app.js, sehingga project tidak membutuhkan database

5. Membuat data awal, Project menggunakan tiga data awal sebagai contoh laporan bencana, yaitu banjir, kebakaran, dan longsor.

Contohnya:

```
let disasterReports = [  
  {  
    id: 1,  
    jenisBencana: "banjir",  
    lokasi: "Kec. Seberang Ulu I",  
    tanggal: "2026-09-20",  
    tingkatKeparahan: "sedang",  
    jumlahPengungsi: 120  
  }  
];
```

ID berikutnya dibuat otomatis menggunakan variabel nextId.

6. Membuat Endpoint GET

(GET /disaster-reports) digunakan untuk mengambil seluruh laporan bencana.

Sedangkan:

(GET /disaster-reports/:id) digunakan untuk mengambil satu laporan berdasarkan ID.

API juga memiliki filter: GET /disaster-reports?jenisBencana=banjir

Filter tersebut digunakan untuk menampilkan laporan yang memiliki jenis bencana tertentu.

7. Membuat Endpoint POST

(POST /disaster-reports) digunakan untuk menambahkan laporan baru.

Contoh request:

```
{  
  "jenisBencana": "gempa",  
  "lokasi": "Kec. Plaju",  
  "tanggal": "2026-10-06",  
  "tingkatKeparahan": "sedang",  
  "jumlahPengungsi": 50  
}
```

Server akan memberikan ID secara otomatis sehingga client tidak perlu menentukan ID.

8. Membuat Endpoint PUT

(PUT /disaster-reports/:id) digunakan untuk memperbarui data laporan secara keseluruhan.

Pada pengujian, data dengan ID 4 diperbarui dari tingkat keparahan **sedang** menjadi **berat** dan jumlah pengungsi menjadi **75 orang**.

9. Membuat Endpoint DELETE

(DELETE /disaster-reports/:id) digunakan untuk menghapus laporan tertentu.

Pada pengujian, laporan dengan ID 4 berhasil dihapus dan server memberikan status 200.

10. Validasi dan Error

API memberikan status 400 apabila field wajib tidak diberikan.

Contohnya request POST yang hanya mengirim:

```
{ "jenisBencana": "gempa", "lokasi": "Kec. Plaju"  
}
```

Server memberikan response:

```
{ "status": 400,  
  "message": "jenisBencana, lokasi, tanggal, dan tingkatKeparahan wajib diisi",  
  "data": null }
```

Untuk ID yang tidak tersedia, server memberikan status 404.

11. Menjalankan API Secara Lokal

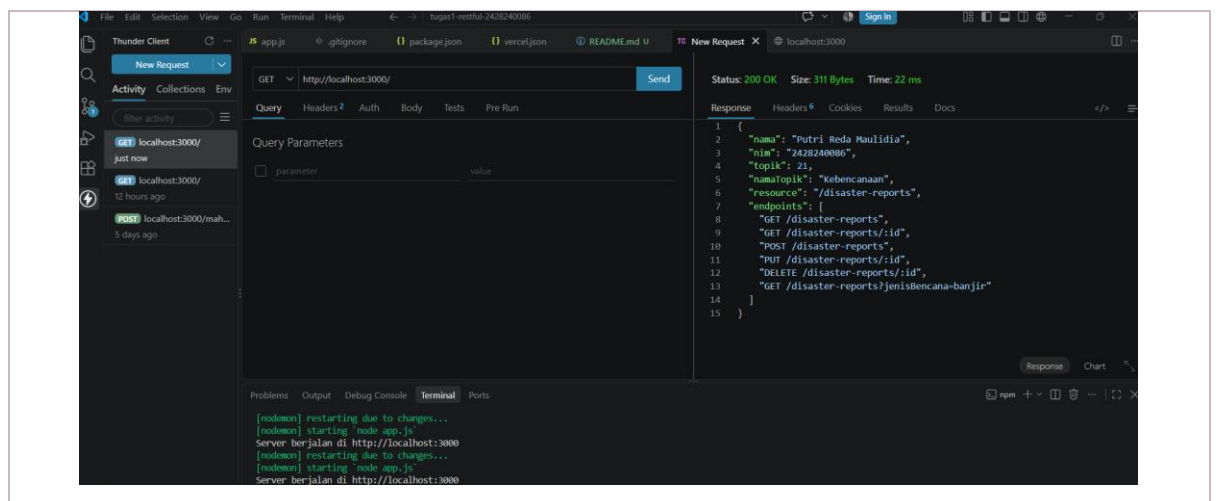
Project dijalankan melalui terminal dengan: **npm install**

kemudian: **npm start**

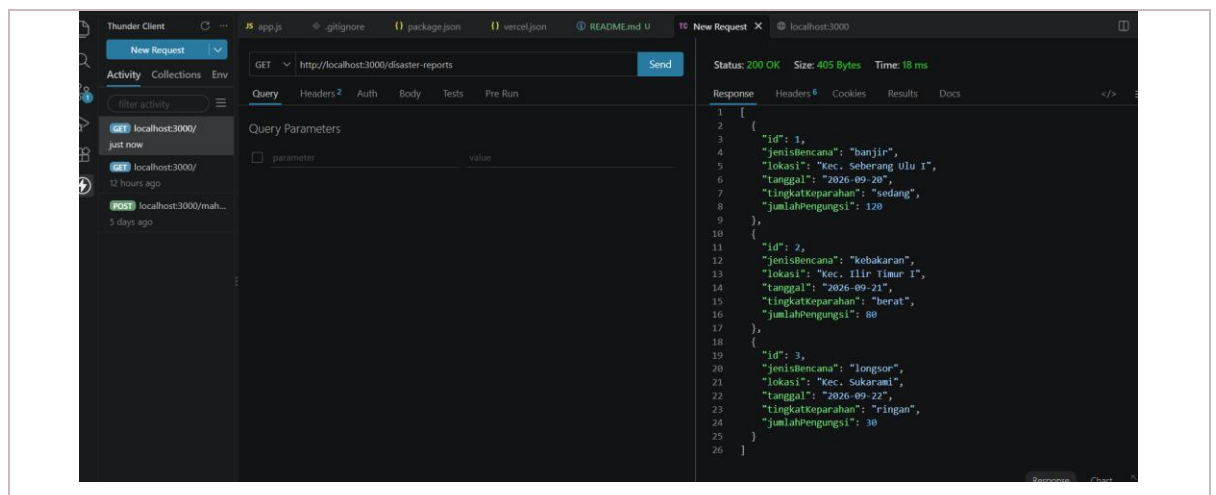
Server berjalan pada: **http://localhost:3000**

Pengujian endpoint dilakukan menggunakan Thunder Client.

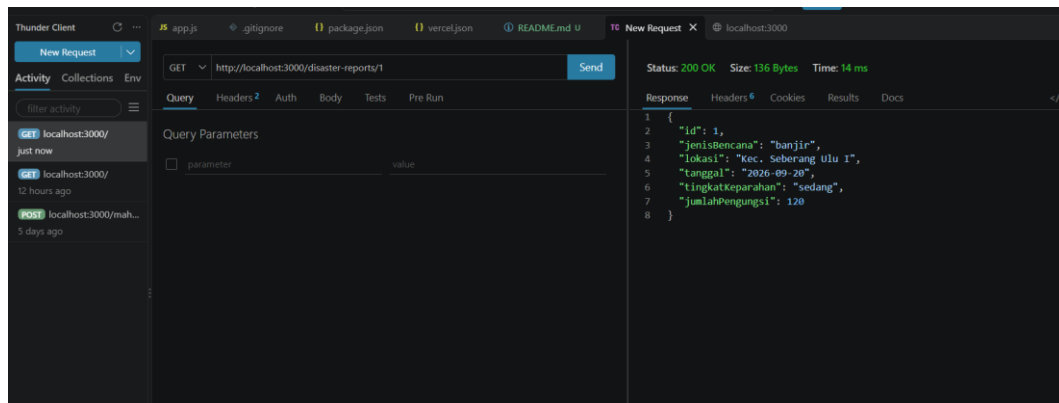
V. Hasil dan Pembahasan



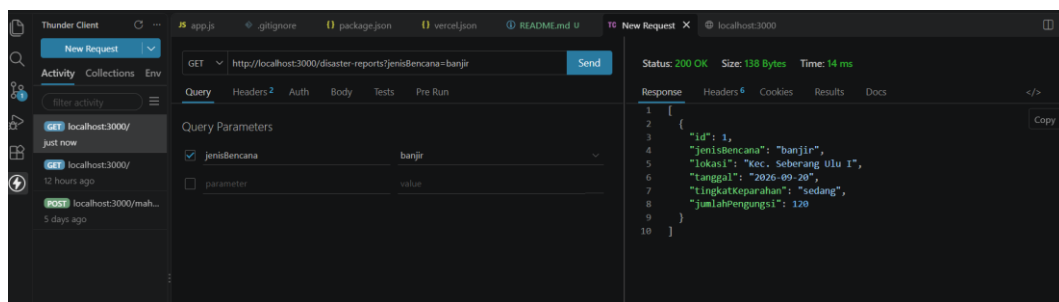
Gambar 1. GET/ <http://localhost:3000/> (Menampilkan halaman awal)



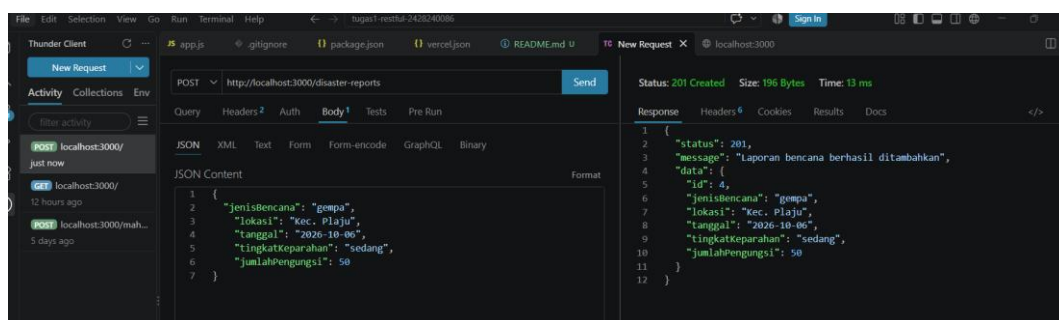
Gambar 2. GET/ <http://localhost:3000/disaster-reports> (menampilkan semua data awal)



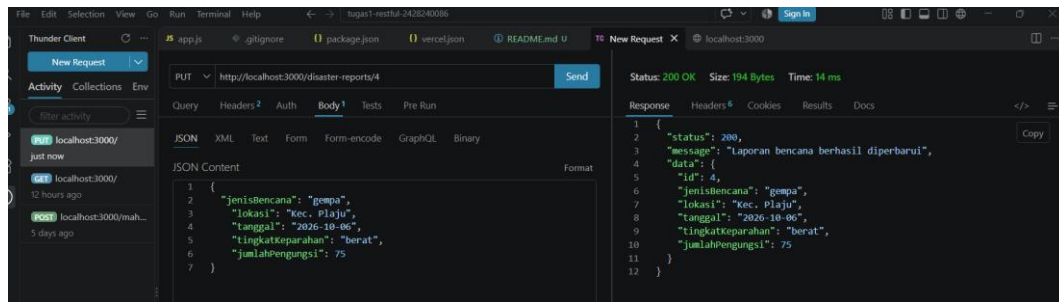
Gambar 3. `GET/ http://localhost:3000/disaster-reports/1` (menampilkan 1 data berdasarkan ID)



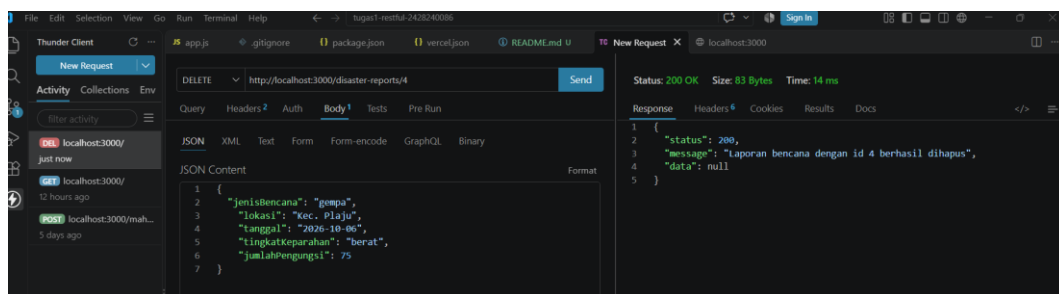
Gambar 4. `GET/ http://localhost:3000/disaster-reports?jenisBencana=banjir` (menampilkan data yang status ya 200)



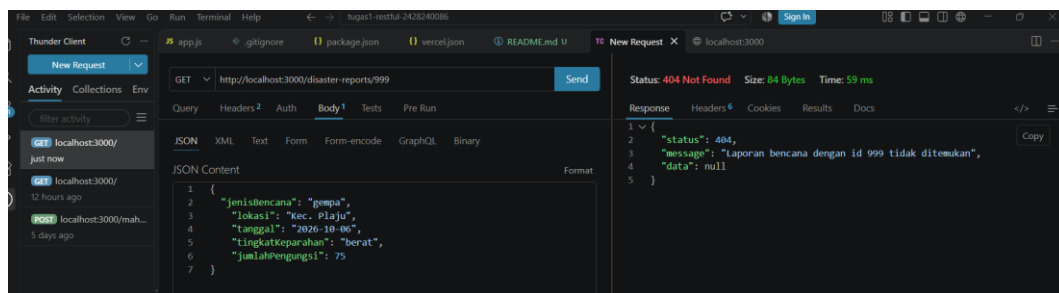
Gambar 5. `POST / http://localhost:3000/disaster-reports` (menampilkan data baru yang ditambahkan)



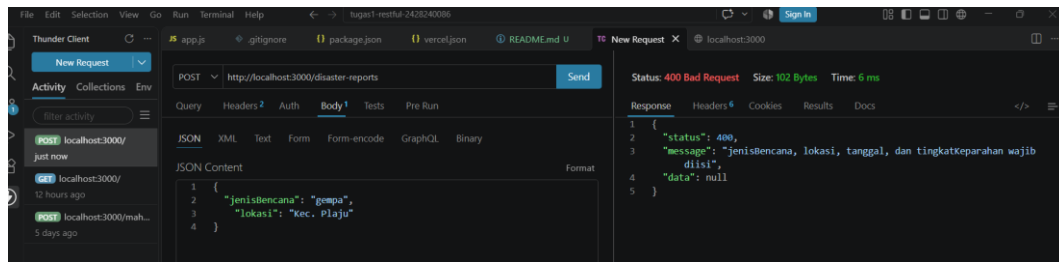
Gambar 6. PUT / <http://localhost:3000/disaster-reports/4> (menampilkan data baru yang sudah diperbarui)



Gambar 7. DELETE / <http://localhost:3000/disaster-reports/4> (menghapus data pada ID 4)



Gambar 8. GET / <http://localhost:3000/disaster-reports/4> (menampilkan data yang tidak ditemukan (404))



Gambar 9. POST / <http://localhost:3000/disaster-reports> (menampilkan data yang error (400))

No.	Method	Endpoint	Status Diharapkan	Status Hasil	Keterangan
1	GET	/disaster-reports	200	200	Berhasil menampilkan seluruh data
2	GET	/disaster-reports/1	200	200	Berhasil menampilkan data berdasarkan ID
3	GET	/disaster-reports?jenisBencana=banjir	200	200	Berhasil melakukan filter berdasarkan jenis bencana
4	POST	/disaster-reports	201	201	Data berhasil ditambahkan
5	PUT	/disaster-reports/4	200	200	Data berhasil diperbarui
6	DELETE	/disaster-reports/4	200	200	Data berhasil dihapus

7	GET	/disaster-reports/999	404	404	ID tidak ditemukan
8	POST	/disaster-reports dengan field kurang	400	400	Validasi berhasil

Tabel 1. Hasil Pengujian pada Endpoint

Pembahasan:

Berdasarkan hasil pembuatan dan pengujian RESTful API menggunakan Express.js, sistem yang dibuat telah menerapkan konsep REST dengan menggunakan resource **/disaster-reports** sebagai pengelolaan data laporan bencana. Data disimpan sementara dalam array sehingga API dapat menjalankan proses tambah, lihat, ubah, dan hapus data tanpa menggunakan database.

Pada proses **GET**, API digunakan untuk mengambil seluruh data laporan bencana maupun mengambil satu data berdasarkan ID. Selain itu, API juga menyediakan fitur filter berdasarkan jenis bencana menggunakan query parameter jenisBencana. Contohnya adalah **GET /disaster-reports?jenisBencana=banjir**, yang hanya menampilkan laporan dengan jenis bencana banjir.

Pada proses **POST**, pengguna dapat menambahkan laporan bencana baru dengan mengirimkan data melalui request body dalam format JSON. ID tidak diberikan oleh pengguna, tetapi dibuat secara otomatis oleh server menggunakan variabel **nextId**. Setelah data berhasil ditambahkan, API memberikan status **201 Created** serta menampilkan data yang baru dibuat.

Pada proses **PUT**, data laporan bencana dapat diperbarui berdasarkan ID. Berbeda dengan POST, PUT digunakan untuk mengganti data yang sudah ada secara lengkap. Jika ID tidak ditemukan, API memberikan status **404 Not Found**, sedangkan apabila terdapat field wajib yang tidak diisi atau nilai yang diberikan tidak sesuai ketentuan, API memberikan status **400 Bad Request**.

Pada proses **DELETE**, data laporan dapat dihapus berdasarkan ID. Jika proses berhasil, API memberikan status **200 OK** dengan pesan bahwa laporan berhasil dihapus. Apabila ID yang dimasukkan tidak ditemukan, API memberikan respons **404 Not Found** dalam format JSON.

Validasi juga diterapkan pada data yang dikirimkan. Field jenisBencana hanya dapat berisi banjir, longsor, kebakaran, atau gempa, sedangkan tingkatKeparahan hanya dapat berisi ringan, sedang, atau berat. Validasi ini bertujuan agar data yang tersimpan tetap sesuai dengan ketentuan yang telah ditentukan.

Hasil pengujian menggunakan **Thunder Client** menunjukkan bahwa seluruh endpoint utama dapat dijalankan sesuai fungsinya. Pengujian mencakup proses menampilkan data, mencari data berdasarkan ID, melakukan filter, menambahkan data, memperbarui data, dan menghapus data. Selain pengujian berhasil, dilakukan juga pengujian kondisi kesalahan seperti ID yang tidak ditemukan dan data yang tidak lengkap. Hasil tersebut menunjukkan bahwa API mampu memberikan status HTTP dan respons JSON yang sesuai.

Secara keseluruhan, RESTful API kebencanaan yang dibuat telah memenuhi konsep dasar RESTful API, yaitu penggunaan endpoint berbasis resource, pemanfaatan metode HTTP sesuai fungsi, penggunaan status code yang sesuai, serta penggunaan JSON sebagai format komunikasi data. API juga tidak menggunakan halaman HTML atau template sehingga sesuai dengan konsep **RESTful API murni** yang menjadi ketentuan tugas.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Perlu memastikan setiap endpoint memberikan status code yang sesuai	Melakukan pengujian setiap method menggunakan Thunder Client

2	Data API disimpan di dalam array sehingga dapat berubah setelah server dijalankan ulang	Menggunakan data awal sebagai data sementara dan memahami bahwa project belum menggunakan database
---	---	--

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, RESTful API dengan Express.js berhasil dibuat untuk mengelola data laporan kebencanaan. Resource /disaster-reports dapat digunakan untuk melakukan operasi pengambilan, penambahan, perubahan, dan penghapusan data.

Penggunaan method GET, POST, PUT, dan DELETE telah diterapkan sesuai fungsi masing-masing. API juga dapat melakukan filter berdasarkan jenisBencana, memberikan ID secara otomatis ketika data baru ditambahkan, serta melakukan validasi terhadap data yang dikirim client.

Hasil pengujian menggunakan Thunder Client menunjukkan bahwa endpoint berhasil memberikan response JSON dengan status code yang sesuai, termasuk status 200, 201, 400, dan 404. Dengan demikian, implementasi dasar RESTful API untuk topik kebencanaan telah berhasil dilakukan.

Daftar Pustaka

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. University of California, Irvine.

MDN Web Docs. (n.d.). *HTTP request methods*. Mozilla.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

MDN Web Docs. (n.d.). *HTTP response status codes*. Mozilla.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

JSON.org. (n.d.). *Introducing JSON*. <https://www.json.org/>

Express.js. (n.d.). *Express - Node.js web application framework*.

<https://expressjs.com/>

Node.js. (n.d.). *Node.js documentation*. <https://nodejs.org/docs/latest/api/>

Lampiran

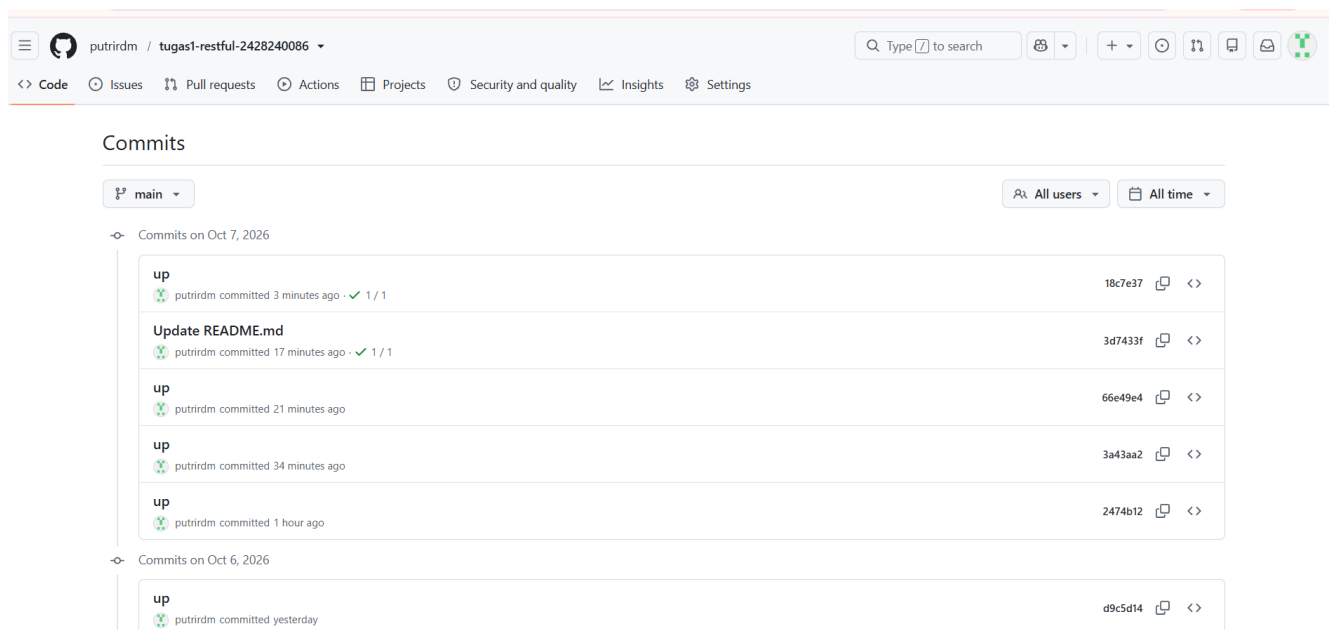
Tautan Repository GitHub:

<https://github.com/putrirdm/tugas1-restful-2428240086>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240086.vercel.app/>

Screenshot Riwayat Commit :



up  putrirdm committed yesterday	4277e1c  <>
up  putrirdm committed yesterday	b6b4677  <>
up  putrirdm committed yesterday	d11783c  <>
Initial commit  putrirdm committed yesterday	510b085  <>